



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A Project Proposal
On
Math2Visual: From Natural Language Prompts to Manim Code for Animated Video
Generation**

Submitted By:

Ashish Kandel (THA078BEI006)
Binita Adhikari (THA078BEI008)
Mukesh Bhatta (THA078BEI022)
Nischal Bhusal (THA078BEI024)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

May, 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, for providing us with the opportunity to undertake this project as part of the requirements for the Bachelor's degree in **Electronics, Communication and Information Engineering**.

We are especially thankful to our respected Supervisor, **Er. Anup Shrestha**, for his continuous support and encouragement throughout the proposal phase of this project.

We would also like to extend our heartfelt thanks to respected Head of Department **Er. Umesh Kanta Ghimire**, the Project Coordinator **Er. Sudip Rana** for their valuable guidance, feedback, and support during the development of the project proposal.

Working on this project will not only strengthen our technical knowledge but also improve our collaboration and problem-solving skills. We look forward to learning and growing throughout the course of this project.

Ashish Kandel (THA078BEI006)

Binita Adhikari (THA078BEI008)

Mukesh Bhatta (THA078BEI022)

Nischal Bhusal (THA078BEI024)

ABSTRACT

Math2Visual is a project aimed at making math education more intuitive by converting plain English math descriptions into animated videos. Our system allows users to type prompts like “plot the graph of $y = \sin(x)$ ” and automatically generates the corresponding Manim code to produce animations. The core of our methodology involves fine-tuning open-source Large Language Models (LLMs) such as Code LLaMA, Mistral, or Phi-2 using a custom dataset of natural language prompts paired with Manim code. This fine-tuned model serves as the backbone for understanding user input and generating Python scripts. The system architecture includes modules for text processing, semantic parsing, code validation, and rendering, all integrated into a simple desktop GUI built with Python frameworks like Gradio or Streamlit. By combining Natural Language Processing (NLP), transformer-based LLMs, and the Manim animation engine, we aim to help students and educators generate dynamic math visuals without needing programming knowledge. Our goal is to bridge the gap between natural language and technical visualization, making mathematical learning more engaging and accessible.

Keywords: *Natural Language Processing (NLP), Manim Animation Engine, Large Language Models (LLMs), Text-to-Code Generation, Mathematics Visualization*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Definition	2
1.4 Objectives	2
1.5 Scope of Project	3
1.6 Application of Project	3
1.7 Limitations	4
2 LITERATURE REVIEW	5
3 REQUIREMENT ANALYSIS	10
3.1 Hardware Requirements	10
3.1.1 High Performance GPU	10
3.2 Software Requirements	10
3.2.1 Python	10
3.2.2 PyTorch	10
3.2.3 Transformers Library (Hugging Face)	11
3.2.4 Manim (Mathematical Animation Engine)	11
3.2.5 CUDA Toolkit and cuDNN (GPU Drivers)	11
3.2.6 Jupyter Notebook or Visual Studio Code (VS Code)	11
3.2.7 FFmpeg	11
3.2.8 Kaggle Notebooks	12
3.2.9 Pandas and NumPy	12
3.2.10 Matplotlib / Seaborn	12
3.2.11 Git and GitHub	12
4 METHODOLOGY	13
4.1 Overview	13

4.2	Dataset Preparation and Model Fine-Tuning	13
4.3	System Architecture	15
4.3.1	User Interface	16
4.3.2	Input Text Processing Module	16
4.3.3	Manim Code Generation Module	17
4.3.4	Manim Code Renderer and Video Generator	18
4.4	System Flowchart	20
4.4.1	Natural Language Prompt	21
4.4.2	Text Understanding (Text Encoder – LLM)	21
4.4.3	Semantic Parser and NL to Code Mapper	21
4.4.4	Code Generator Module (Generates Manim Code)	22
4.4.5	Code Validator and Syntax Checker	22
4.4.6	Manim Renderer Engine	22
4.4.7	Output Video File	23
4.5	Tools and Technologies Component	24
4.6	Large Language Models (LLMs)	24
4.6.1	Basic Architecture of LLMs	25
4.6.2	How LLMs Learn from Data	27
4.6.3	Role of LLMs in Math2Visual	27
4.7	Hugging Face Transformers	28
4.7.1	Evaluation of Hugging Face Transformers for LLM-Based Applications	28
4.8	Code LLaMA	30
4.8.1	Architecture and Sizes	30
4.8.2	Training Data	30
4.8.3	Role of Code LLaMA in Our Project	31
4.9	Mistral	31
4.10	Phi-2	32
4.10.1	Design Philosophy	33
4.10.2	Role of Phi-2 helps in our project	33
4.11	Manim	33
4.12	GUI Implementation	34
4.12.1	Python GUI	35
4.13	Evaluation and Testing Strategy	35
4.13.1	Code Accuracy	35
4.13.2	Animation Correctness	35
4.13.3	Latency	36
4.13.4	Testing Metrics	36

4.13.5	Test Cases	39
5	FEASIBILITY ANALYSIS	40
5.1	Economic Feasibility	40
5.2	Technical Feasibility	40
5.3	Operational Feasibility	40
5.4	Ethical Feasibility	40
6	EXPECTED OUTCOME	41
6.1	Prompt vs. Generated Code Example	41
6.1.1	Input Prompt	41
6.1.2	Mathematical Interpretation	41
6.1.3	Generated Manim Code	41
6.1.4	Generated video after compilation of code	43
6.1.5	Wireframe	44
7	APPENDICES	45
	Appendix A: Gantt Chart	45
	Appendix B: Expected Budget	46
	Appendix C: Mathematical Foundation of Transformer-Based LLMs	47

List of Figures

Figure 4-1: Proposed System Block diagram	15
Figure 4-2: Proposed System Flowchart diagram	20
Figure 6-1: Rendered output of the animation generated from prompt	43
Figure 6-2: Wireframe of the Math2Visual interface showing the code generation panel	44
Figure 6-3: Wireframe of the Math2Visual interface displaying the video playback panel	44
Figure 7-1: Gantt chart	45

List of Tables

Table 2-1: Summary of Literature Review	9
Table 4-1: Tools and Technologies	24
Table 4-2: Example Test Cases for Evaluation	39
Table 7-1: Budget Estimation of the project	46

List of Abbreviations

3D	Three-Dimensional
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
BLEU	Bilingual Evaluation Understudy
CUDA	Compute Unified Device Architecture
CV	Computer Vision
DL	Deep Learning
DNN	Deep Neural Network
FFmpeg	Fast Forward Moving Picture Experts Group
GPT	Generative Pre-training Transformer
GPU	Graphical Processing Unit
HF	Hugging Face
IEEE	Institute of Electrical and Electronics Engineers
JSON	Javascript Object Notation
LLM	Large Language Model
Manim	Mathematical Animation Engine
ML	Machine Learning
NL2Code	Natural Language to Code
NLP	Natural Language Processing
NLU	Natural Language Understanding
NN	Neural Network
RLHF	Reinforcement Learning with Human Feedback
SOTA	State-of-the-art
SVG	Scalable Vector Graphics
UI	User Interface

1. INTRODUCTION

Mathematics is a foundational subject in engineering, science, and technology, but many students find it difficult to understand because of its abstract nature and complex symbols. Traditional teaching methods often fail to help students build a clear mental picture of mathematical ideas. To solve this problem, our project aims to develop an AI-powered system that takes a math-related sentence written in natural language and converts it into Manim code. When this code is compiled, it produces an animated video that visually explains the given concept. This system will help students learn math more easily by turning difficult equations or descriptions into clear, engaging animations.

1.1. Background

Mathematical visualization plays a crucial role in enhancing comprehension, problem-solving, and retention. Recent educational studies emphasize that learners, particularly in STEM fields, benefit significantly from graphical and animated representations of abstract concepts. Tools like Manim (Mathematical Animation Engine) have emerged as powerful platforms for creating high-quality animations of mathematical phenomena. However, Manim's steep learning curve and its dependency on Python scripting limit its accessibility to educators and learners without programming experience.

In parallel, advancements in Natural Language Processing (NLP) and large language models (LLMs) have enabled systems to understand and generate human-like text, including executable code. These technologies can serve as a bridge between user-friendly language input and complex backend tools like Manim. By leveraging NLP and AI-driven code generation, it is now feasible to create systems that understand a user's intent and automatically produce the corresponding animation scripts, reducing the technical burden and enabling a wider audience to visualize mathematical ideas effectively.

1.2. Motivation

Even though many online platforms and tools exist for learning mathematics, very few can turn mathematical ideas into dynamic animations without needing programming skills. Most systems require users to manually write animation scripts or depend on static images and pre-recorded videos, which often don't match the exact concept being taught. This makes it difficult for both students and educators to visualize important topics such as functions, geometric shapes, algebraic transformations, and calculus-based change especially when those topics involve continuous motion or change.

Traditionally, math visualizations are created using tools like GeoGebra, Desmos, or by manually coding animations using engines like Manim. While these tools are powerful, they either lack flexibility in animation or require technical knowledge that not all learners or teachers possess. As a result, much time is spent either learning to code or finding pre-made content, which limits creativity and accessibility in classrooms.

This project is inspired by the idea of making mathematics more interactive and intuitive using the power of artificial intelligence. By allowing users to type math-related prompts in plain English and automatically generating Manim code that produces visual animations, we aim to remove the technical barrier. This approach brings together natural language processing (NLP) and programmatic visualization, creating a smart, user-friendly tool that can assist learners, teachers, and content creators alike.

In a world where technology is rapidly transforming education, this project offers a new way to engage with mathematical content visually, helping to bridge the gap between symbolic math and conceptual understanding. It has the potential to improve math learning outcomes, especially for students who struggle with abstract thinking or who learn better through visual means.

1.3. Problem Definition

Understanding mathematics can be difficult for many learners, especially when it involves abstract ideas and complex symbols. Most existing math learning tools rely heavily on static images, formulas, and text-based explanations. These methods often do not provide the visual clarity that students need to fully grasp mathematical concepts. Moreover, manually creating visual animations for math content requires programming skills, time, and effort, which is not practical for all learners or educators.

There is currently no widely available system that allows users to simply type math-related ideas in natural language and automatically receive meaningful animated visualizations. This lack of intuitive tools makes it harder for students to build a strong conceptual foundation in mathematics and slowly lose the interest in subject as well.

1.4. Objectives

The main objectives of proposed project are:

1. To develop a Natural Language Processing module that can accurately interpret mathematical prompts written in plain English and convert them into corresponding Manim code.

2. To implement a system that compiles the generated Manim code and renders it as an animated video to visually represent the described mathematical concept.

1.5. Scope of Project

This project focuses on developing an AI-powered software system that transforms plain English descriptions of mathematical concepts into animated visualizations. The system will utilize natural language processing techniques to interpret user input and automatically generate Manim (Mathematical Animation Engine) code. This code will then be compiled to create animations that help visually convey the underlying mathematical ideas. The project will support a range of fundamental mathematical topics such as geometry, algebra, equations, and graph-based concepts. It will feature a user-friendly interface that allows users to input natural language prompts and view the resulting animations. The backend will handle language understanding, code generation, and rendering, forming a complete pipeline from text to animation. By combining NLP and visual representation through animation, the system aims to make learning mathematics more intuitive, engaging, and accessible, especially for students and educators in academic settings.

1.6. Application of Project

1. **Interactive Math Learning** : Transforms written math descriptions into animations, making abstract topics easier to visualize and grasp for students.
2. **Smart Content Creation for Educators** : Helps teachers and tutors quickly generate custom visualizations for lectures, tutorials, and presentations using simple English instructions.
3. **Test Preparation and Concept Review Tools** : Used in test-prep apps (e.g. IOE, SAT, GRE, JEE) to generate on-the-fly visual explanations for practice problems or review sessions.
4. **Integration with AR/VR Learning Environments** : The generated Manim outputs could be extended into augmented or virtual reality settings to immerse learners in 3D mathematical environments.

1.7. Limitations

1. The system supports only basic mathematical topics like geometry, algebra, simple graphs, and equations. Advanced topics such as calculus or differential equations are not included.
2. Natural language processing works best with clear and simple English prompts; ambiguous or complex sentences may cause incorrect Manim code generation.
3. The system currently supports only English text input and does not handle multiple languages or voice commands.

2. LITERATURE REVIEW

Mathematics education significantly benefits from visualization, as it bridges the gap between abstract concepts and intuitive understanding. Visualization, especially through animations, has proven effective in enhancing students grasp of complex mathematical concepts by offering dynamic representations that static images and texts often fail to convey [1]. Animated visuals can portray intricate mathematical transformations, spatial relationships, and functions dynamically, thus improving comprehension, engagement, and retention among learners [2]. This pedagogical effectiveness underscores the need for accessible, intuitive tools that facilitate math animations without requiring extensive technical knowledge from educators or students. .

Currently, several frameworks and tools exist for mathematical visualization, notably Manim, GeoGebra, and Desmos. Manim, developed initially by Grant Sanderson (3Blue1Brown), is highly praised for producing professional-quality math animations used extensively in educational YouTube videos [3]. However, despite its powerful capabilities, Manim’s accessibility remains limited due to its steep learning curve, as it requires proficiency in Python programming and familiarity with its scripting style [4]. GeoGebra and Desmos offer user-friendly graphical interfaces but generally lack the dynamic animation scripting capabilities that Manim provides. Thus, there exists a clear gap between ease of use and the production of detailed, custom animated visuals.

Advances in natural language processing (NLP) have enabled substantial improvements in educational technology interfaces. Symbolic mathematics libraries, like SymPy, combined with NLP, can interpret human language descriptions into structured symbolic expressions or mathematical operations [5]. Furthermore, tools leveraging large language models (LLMs) such as GPT, Code LLaMA, and Phi have shown promising results in translating natural language queries into executable mathematical code [6]. For instance, the integration of OpenAI’s GPT models with mathematical engines allows users to formulate complex mathematical questions in natural language, significantly reducing barriers for those without strong programming backgrounds [7].

Large language models have increasingly gained traction in automating code generation tasks, substantially impacting software engineering practices. Models like GPT-4, LLaMA, Mistral, Phi-2, and Falcon exhibit strong performance in generating syntactically correct and contextually accurate code snippets from textual descriptions [8]. Research by Chen et al. on OpenAI's Codex demonstrated remarkable capabilities in Python code synthesis, significantly outperforming earlier models in tasks involving converting natural language into executable code [9]. Likewise, Meta's Code LLaMA and Microsoft's Phi-2 have set benchmarks in open-source code generation, proving their utility in real-world software development scenarios [10]. These advances demonstrate the practicality of employing LLMs in automated mathematical visualizations, indicating potential integration with animation libraries such as Manim.

Several studies have explored translating natural language into executable mathematical code. Hendrycks et al. presented transformer-based methods for code generation tasks, specifically emphasizing Python and JavaScript, showcasing that transformer models significantly outperform previous rule-based or template-driven approaches [11]. Similarly, research efforts leveraging Hugging Face Transformers highlighted effective fine-tuning techniques for generating domain-specific mathematical code [12].

However, relatively few studies directly connect natural language prompts with complex mathematical animation scripts. Initial attempts to bridge this gap often resulted in either limited scope or reliance on predefined templates, thereby reducing generalizability and flexibility [13]. For instance, existing tools might translate simple phrases to basic animations but fail to handle intricate mathematical instructions or provide meaningful interactivity or adaptability to user needs.

Fareed Khan's project, AI Text to Video Model from Scratch, offers a foundational approach to generating video content directly from textual prompts using deep learning techniques. Built entirely in Python, the model utilizes Generative Adversarial Networks (GANs) to translate descriptive sentences into corresponding video sequences. The work involves essential stages such as data preprocessing, text embedding using Word2Vec, and constructing a GAN-based architecture to handle the generation process. Although the system is lightweight and intended for educational purposes, it effectively demonstrates the feasibility of aligning textual semantics with visual content generation. This project provides valuable insights into the core mechanics of AI-powered video generation and serves as a practical reference for developing systems like Math2Visual, which aim to transform natural language math descriptions into automated, educational animations [14].

The Text2Scene model by Tan et al. [15] introduces a sequential method for generating visual scenes from text by predicting and placing objects with specific attributes like size, position, and appearance. Unlike end-to-end image generators, it offers interpretable, compositional outputs an approach that aligns well with the structured nature of educational animations. For Math2Visual, this method supports the idea of breaking down mathematical prompts into individual visual elements, making it a relevant reference for text-to-animation systems.

OpenAI’s model, Sora, presents a diffusion transformer architecture capable of generating high-definition, minute-long videos directly from textual descriptions. According to OpenAI’s documentation and technical details, Sora focuses on maintaining temporal consistency and simulating realistic physical interactions, although it still faces challenges in modeling complex causal reasoning and real-world physics [16].

Similarly, Google DeepMind’s Veo model represents a multimodal generative system that produces 1080p videos of over a minute duration. As outlined in recent public documentation, Veo integrates audio with video and allows fine-grained editing. Its third version (released in 2025) includes synchronized audio capabilities, illustrating the potential of aligning language, visual scenes, and audio streams in educational video generation [17]. Dream Machine by Luma AI introduces a lightweight solution capable of producing five-second videos from either text or image prompts. It emphasizes motion realism and camera perspective simulation. While praised for its smooth rendering of human gestures and cinematic movement, critiques have emerged regarding its lack of transparency in training data and model source details [18].

The reviewed literature reveals notable limitations in current approaches, especially in automating the generation of professional-quality animations directly from natural language descriptions without extensive coding expertise. Although NLP and LLMs demonstrate potential in simplifying code generation, their application in specialized math animation domains remains underexplored. Additionally, the existing animation frameworks, notably Manim, have not been effectively integrated with NLP and LLM-based code generation methodologies in an accessible, user-friendly interface.

Furthermore, comparative evaluations of multiple LLMs in terms of animation quality, code correctness, and execution efficiency in generating complex mathematical animations are scarce. This gap indicates an opportunity for comprehensive research and practical implementation to enhance user accessibility and productivity in educational technology.

Building upon the identified research gaps, the proposed ”Math2Visual” project aims to develop a lightweight, intuitive system leveraging open-source LLMs (e.g., Code LLaMA, Mistral, Phi-2) to translate natural language prompts directly into executable Manim anima-

tions. This project uniquely addresses the accessibility issue by providing a Python GUI interface (via Tkinter or PyQt), facilitating interaction for users lacking programming skills. By systematically comparing multiple LLM outputs based on accuracy, rendering quality, and response efficiency, this project aspires to set a practical benchmark for future developments in AI-assisted educational visualization tools.

Table 2-1: Summary of Literature Review

Citation	Methods	Findings and Conclusion
Arcavi (2003)	Review of visual representation in math education	Visuals enhance understanding and bridge abstract concepts; essential in math learning.
Mayer (2014)	Cognitive theory on multimedia learning	Animations improve comprehension and retention by engaging cognitive processes.
3Blue1Brown (2020)	Documentation on Manim	Manim enables professional-quality math animations but requires coding skills.
ManimCommunity (2023)	Open-source development of Manim	Offers high-quality output but limited by Python proficiency requirement.
Meurer et al. (2017)	SymPy for symbolic computing in Python	Enables math expression parsing, supporting natural language interfaces.
Radford et al. (2021)	LLM-based visual model training	LLMs can learn transferable understanding from text for code generation.
OpenAI (2021)	Codex model for code generation	Supports converting natural language into Python code for math tasks.
Tunstall et al. (2022)	Overview of NLP with transformers	Transformers are key to modern NLP and code generation techniques.
Chen et al. (2021)	Evaluation of Codex on code generation	Codex outperforms prior models in natural language to code tasks.
Meta AI (2023)	Code LLaMA development	Open-source code generation with improved accuracy and accessibility.
Hendrycks et al. (2021)	Transformers for code generation	Transformers significantly outperform rule-based methods for code synthesis.
Hugging Face (2023)	Fine-tuning transformers	Domain-specific fine-tuning boosts accuracy in generating math-related code.

3. REQUIREMENT ANALYSIS

This section outlines fundamental hardware and software prerequisites essential for the successful execution and completion of this project.

3.1. Hardware Requirements

3.1.1. High Performance GPU

A high-performance GPU is crucial for fine-tuning large language models like Code LLaMA and Mistral, which contain billions of parameters. These models require immense parallel processing power that is only feasible with modern GPUs. GPUs significantly reduce the training time, allow larger batch sizes, and support operations like attention mechanisms and matrix multiplications at high speed. Without a capable GPU, the model training would be extremely slow or even infeasible. Additionally, GPU acceleration is necessary during inference to provide fast, real-time code generation based on user prompts.

3.2. Software Requirements

3.2.1. Python

Python is the backbone of this project, providing the primary language for all development work. It enables seamless integration of machine learning libraries (like PyTorch and Hugging Face Transformers) and visualization tools (like Manim). The Python ecosystem also includes utility libraries for data manipulation, training visualization, and prompt preprocessing. Version 3.10 or higher is recommended for compatibility with the latest features in these frameworks.

3.2.2. PyTorch

PyTorch is the deep learning framework used to fine-tune and evaluate large language models in this project. It supports GPU acceleration, dynamic computational graphs, and rich debugging capabilities. PyTorch is compatible with the Hugging Face Transformers library, enabling straightforward integration of LLMs like Code LLaMA and Mistral. It is widely adopted in the research community and known for its flexibility and ease of use in both training and inference stages.

3.2.3. Transformers Library (Hugging Face)

The Hugging Face Transformers library provides access to pre-trained language models, tokenizers, and training utilities. It simplifies the process of fine-tuning and evaluating models using high-level APIs. The library also includes tools for managing datasets, tracking training progress, and exporting models for deployment. It forms the core of the model interaction layer in this project, enabling input prompts to be processed and corresponding code to be generated.

3.2.4. Manim (Mathematical Animation Engine)

Manim is an open-sourced Python-based engine used to create animated mathematical visualizations originally developed by Grant Sanderson popularly known as 3blue1brown. In our project, manim converts the model-generated Python code into explanatory videos. Manim allows frame-by-frame control of animations, supports LaTeX for mathematical notation, and includes a wide range of classes for geometrical and graphical visualization. It bridges the gap between abstract code and visual learning by producing professional-quality educational animations from user prompts.

3.2.5. CUDA Toolkit and cuDNN (GPU Drivers)

For GPU acceleration to work with PyTorch and Hugging Face models, the CUDA Toolkit and cuDNN drivers must be installed. These tools provide the low-level hardware support needed to utilize the full computational power of NVIDIA GPUs. They ensure compatibility between software libraries and GPU hardware, enabling efficient parallel computation, matrix multiplication, and deep learning operations on massive datasets.

3.2.6. Jupyter Notebook or Visual Studio Code (VS Code)

These integrated development environments (IDEs) are suitable for writing, testing, and debugging code. Jupyter is ideal for rapid prototyping and experimentation, while VS Code offers a full development suite with Git integration and debugging tools. Both environments support live visualization, markdown annotations, and plugins for Python, Manim, and AI development, making them ideal for iterative development in this project.

3.2.7. FFmpeg

FFmpeg is an essential component in our project that handles the final step of transforming raw animations into proper video files. After Manim renders the animation as a sequence

of individual image frames, FFmpeg takes those frames and stitches them together to create a continuous video. Internally, it uses video encoding techniques (like H.264) to compress the frames efficiently and outputs the result in widely supported formats such as MP4. In our system, once the Manim code is generated from the user's natural language prompt and the rendering is complete, FFmpeg automatically compiles the output into an MP4 video, making it ready for download or playback. This step is what allows the animations created by our tool to be saved, shared, and reused easily in classrooms or online platforms.

3.2.8. Kaggle Notebooks

Kaggle provides a free, cloud-based environment with GPU support, making it ideal for model training and experimentation when local resources are limited. It supports collaborative workflows, public datasets, and version tracking via notebooks, aiding reproducibility and benchmarking.

3.2.9. Pandas and NumPy

These libraries are essential for preprocessing datasets, handling structured inputs/outputs, and computing metrics during training and evaluation. They streamline tasks such as prompt formatting, result comparison, and loss tracking.

3.2.10. Matplotlib / Seaborn

These visualization tools help plot training losses, evaluation metrics, and model comparison graphs. They provide insights into model performance and help guide future fine-tuning iterations.

3.2.11. Git and GitHub

Version control using Git and online repositories like GitHub helps collaborate with team, track code changes and manage training experiments. It is especially helpful for storing model checkpoints, code versions, and training logs. GitHub also provides issue tracking, continuous integration, and a platform for sharing your final project with the community or team members.

4. METHODOLOGY

4.1. Overview

This project proposes an AI-powered pipeline that converts natural language prompts into executable Manim code and renders the corresponding animated mathematical videos. The system utilizes fine-tuned large language models (LLMs) to automate code generation and embeds this process into a user-friendly Python-based graphical interface. The goal is to enhance accessibility to mathematical visualization without requiring programming knowledge.

4.2. Dataset Preparation and Model Fine-Tuning

To enable accurate conversion of natural language to Manim code, a custom dataset of prompt-code pairs will be prepared. This dataset will consist of:

- Manually curated examples of mathematical topics (e.g., graphing, equations, transformations) and their corresponding Manim code.
- Synthetic examples generated using templates and variations to expand coverage.

Templates here refer to pre-defined natural language prompt structures paired with reusable Manim code patterns. For example, a prompt like “Plot the graph of $f(x) = x^2$ ” maps to a general code template that plots any function using Manim’s `plot()` method. By substituting variables and expressions in these templates, we generate a wide variety of synthetic examples for training.

Manual curation involves carefully designing and reviewing prompt-code pairs to ensure semantic accuracy and alignment. These examples are handcrafted based on real math topics (e.g., transformations, derivatives) and serve as ground truth for model fine-tuning. Similar techniques have been applied in datasets like HumanEval (OpenAI) and MBPP (Google AI).

- Validation and test sets to evaluate generalization across math topics and input phrasing.
- Multiple open-source LLMs, including LLaMA, Mistral, or Phi-2, will be fine-tuned using these datasets. The fine-tuning will be performed using Hugging Face Transformers and DeepSpeed to optimize code generation performance.

An evaluation strategy will be applied to compare these models based on predefined criteria such as:

1. **Code correctness:** Whether the generated Manim code runs without errors.
2. **Response time:** Inference speed under consistent test conditions.
3. **Animation quality:** Accuracy and clarity of the rendered mathematical concept.

The model achieving the highest overall performance across these metrics will be selected for final integration into the system.

Example Prompt–Code Template Pair:

- **Prompt Template:** “Plot the graph of $f(x) = ___$ ”
- **Code Template:**

```
1 class PlotGraph(Scene):  
2     def construct(self):  
3         ax = Axes(...)  
4         graph = ax.plot(lambda x: ..., ...)  
5         self.play(Create(graph))
```

Listing 1: Function Plot Template in Manim

By replacing the placeholder expression with functions like x^2 , $\sin(x)$, or e^x , we can generate a wide range of mathematical visualization examples automatically.

4.3. System Architecture

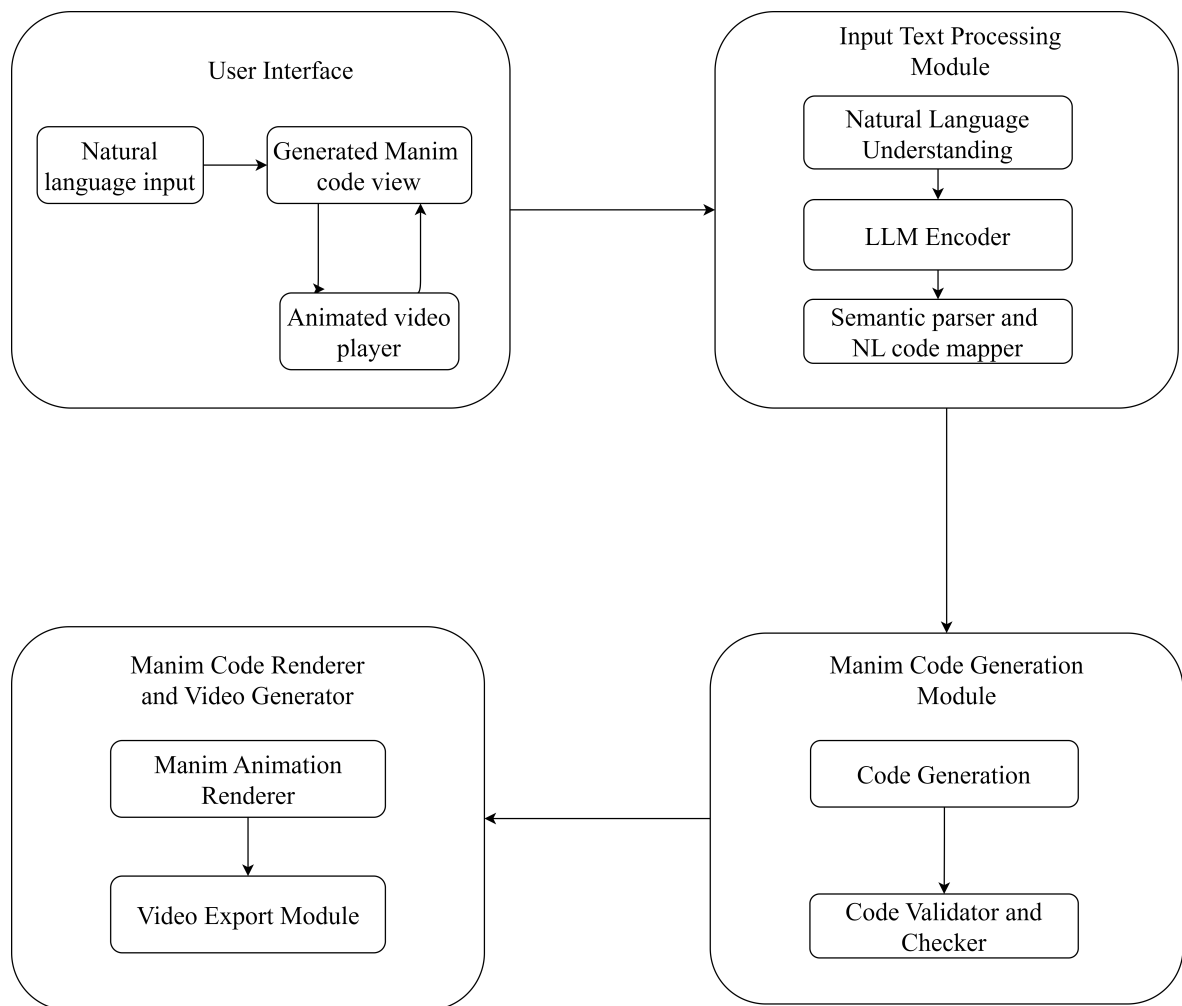


Figure 4-1: Proposed System Block diagram

The overall architecture of our proposed system is purposefully designed to translate plain English user input into high-quality mathematical animations, using the power of large language models (LLMs) and the Manim animation library. The architecture is carefully divided into four major modules that interact seamlessly to ensure a robust, flexible, and user-friendly workflow from start to finish. Below, each major block is described in detail, providing a clear understanding of its role and contribution to the complete pipeline.

4.3.1. User Interface

The user interface (UI) is the main touchpoint for the end user and is arguably the most important aspect from a usability perspective. Its design ensures that anyone, regardless of programming background, can use the system intuitively and efficiently.

- **Natural Language Input:** At the very start, users interact with a simple and clear input field where they can describe the animation they want using regular English sentences. For example, a mathematics teacher can enter, “Show how to multiply two matrices visually,” or a student might request, “Draw a graph of $y = \sin(x)$ and animate its transformation into $y = \cos(x)$.” This approach lowers the barrier to entry for non-coders and empowers a broader audience to use advanced visualization tools without technical hurdles.
- **Generated Manim Code View:** After processing the prompt, the system transparently displays the automatically generated Manim Python code to the user. This serves several purposes: it helps users see how their words are interpreted into code, provides an opportunity for those familiar with Manim or Python to tweak or learn from the code, and promotes trust in the system by making the “black box” translation process visible. Advanced users can even make small edits to customize the animation before rendering.
- **Animated Video Player:** Once the code is generated and validated, the UI includes an embedded player to immediately show the resulting animation. This feature is invaluable for instant feedback: users can quickly verify if the output matches their expectations, rerun or adjust their prompts as needed, and download or share the final video. Such an interactive, looped design keeps the user in control at all times and makes experimentation and learning easy.

4.3.2. Input Text Processing Module

This module acts as the core intelligence of the system, responsible for parsing, understanding, and encoding the meaning behind the user’s prompt. Its layered design provides both

flexibility and precision.

- **Natural Language Understanding (NLU):** This sub-module is responsible for deeply analyzing the user’s sentence. Instead of simply matching keywords, it understands grammatical structure, relationships, and mathematical context. For instance, in “Animate a triangle rotating about its centroid,” the NLU extracts not just the geometric object (triangle) but also identifies the action (rotation), the center of rotation (centroid), and any additional numerical parameters (angle, direction, etc.). This enables nuanced, context-aware comprehension far beyond rule-based systems.
- **LLM Encoder:** At this point, a pre-trained large language model (LLM) such as LLaMA, Mistral, Phi, or a similar transformer-based architecture encodes the user’s intent into a dense, high-dimensional vector. This representation captures semantic nuances, synonyms, implicit relationships, and even “hidden” details that are not stated directly but are often assumed in mathematical communication. Such deep encoding allows the system to generalize to new prompts, deal with ambiguous phrasing, and understand more complex or creative requests.
- **Semantic Parser and NL Code Mapper:** Leveraging the contextual encoding, this component translates the user’s natural language instructions into a structured, step-by-step logical plan that directly corresponds to Manim’s API and coding conventions. It decomposes complex instructions (“Draw a circle, color it red, and make it shrink and expand repeatedly”) into discrete actions, maps those to Manim objects and methods, and preserves dependencies between steps. This ensures both fidelity to the user’s intent and technical feasibility in the code generation phase.

4.3.3. Manim Code Generation Module

This is the module where the user’s idea truly becomes real, the structured representation is converted into actual executable code that Manim can use to generate animations.

- **Code Generation:** The generator takes the parsed logic and systematically builds Python code that adheres to Manim’s syntax and object model. It imports required libraries, defines necessary variables, creates objects (like circles, graphs, vectors), and adds animation sequences as specified by the prompt. The code generator intelligently handles multiple objects, layered animations, and custom user modifications (if any), ensuring the code is not just correct, but also efficient and readable. This step bridges high-level human ideas with the technical demands of animation scripting.

- **Code Validator and Checker:** Before proceeding to rendering, the generated code passes through a stringent validation phase. Here, both Python and Manim-specific errors are caught early: be it syntax issues, undefined variables, wrong function calls, or unsupported features. In case of errors, the validator can provide user-friendly feedback or even suggest fixes. This quality control ensures system stability, reduces failed renders, and improves the reliability of the whole workflow.

4.3.4. Manim Code Renderer and Video Generator

The final module is responsible for transforming the validated code into tangible, shareable output.

- **Manim Animation Renderer:** The renderer executes the validated Manim code in a sandboxed environment, frame by frame, using Manim’s powerful vector graphics engine. It manages timing, object layering, transitions, and camera movements, guaranteeing a polished visual output. Even for complicated scenes, like parametric curves, geometric constructions, or stepwise proofs, the renderer ensures the animation remains smooth and visually clear.
- **Video Export Module:** After rendering, the generated frames are compiled into a standard video file (e.g., MP4), ready for use in presentations, online platforms, or personal study. The module allows users to select different resolutions, formats, and even frame rates depending on their needs. The final video is instantly available for viewing in the UI and for download or sharing, supporting seamless integration with other digital tools or learning environments.

System Integration and Workflow:

Each module in the system is designed to function both independently and as part of a tightly integrated pipeline. Data flows smoothly between modules, with clear feedback loops allowing the user to modify prompts or code at any stage if necessary. The modular design allows for future upgrades such as plugging in a more advanced LLM, adding new animation templates, or supporting additional output formats without requiring major changes to the rest of the system.

Why This Architecture?

This modular, stepwise architecture is chosen for its transparency, flexibility, and user centric approach. By breaking the process into logical blocks, it’s easier to isolate issues, improve individual components, and scale the system as user needs grow. It also makes the technology

accessible: even users unfamiliar with coding or Manim can produce complex animations, while advanced users can engage at a deeper level if they wish.

In summary, our system architecture delivers a highly accessible and technically robust workflow for generating mathematical animations from plain English descriptions. Through careful design and seamless integration, it brings together the latest in AI language models and visualization technology to democratize the creation of visual educational content in our project.

4.4. System Flowchart

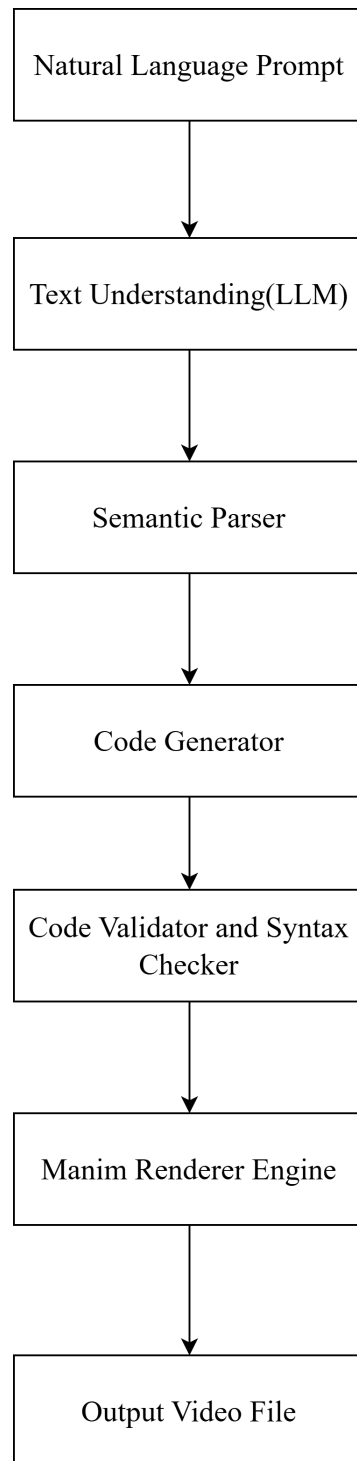


Figure 4-2: Proposed System Flowchart diagram

The overall workflow of our system is depicted in the flowchart above, and it outlines each critical phase in transforming a simple user prompt into a fully animated video. This flowchart outlines the main modules involved in converting a user’s natural language prompt into a final mathematical animation video in our project. Each stage is designed to keep the process simple for the user, while ensuring the technical steps are robust and reliable.

4.4.1. Natural Language Prompt

At the very beginning, our system invites the user to type in their mathematical concept or visualization idea using plain, english language. This input could be anything from “Draw a unit circle and label the axes” to “Animate the multiplication of two matrices step by step.” The main aim here is to make advanced visualization accessible even for those with zero programming background. By lowering the entry barrier, we enable teachers, students, and self-learners to describe their needs without wrestling with any code or technical jargon. Each prompt serves as a blueprint for our system, capturing the user’s intent in its raw, intuitive form.

4.4.2. Text Understanding (Text Encoder – LLM)

Once the user submits their prompt, the real magic begins with the Text Understanding module, which is powered by a modern Large Language Model (LLM). This AI-driven component does not simply tokenize words; it reads the sentence almost like a human would—identifying mathematical terms, actions, objects, and even the relationships between them. For example, in the phrase “Rotate the triangle by 90 degrees about its centroid,” it can pick out the main entity (triangle), the required transformation (rotation), the angle (90 degrees), and the reference point (centroid). This level of comprehension is crucial because it transforms ambiguous human language into a structured representation that machines can actually work with. The quality and intelligence of this step directly determine how accurately the system can fulfill complex user requests, especially when the language is informal or open-ended.

4.4.3. Semantic Parser and NL to Code Mapper

After the LLM deciphers the user’s intent, the semantic parser takes over. We can think of this as a translator that bridges the gap between natural language and formal code logic. This module doesn’t just do one-to-one word mapping; it interprets context, disambiguates similar terms (like “plot” versus “draw”), and builds connections between user concepts and Manim’s object-oriented constructs. For example, if a user says “show a vector from origin to (3,4),” the parser figures out that it needs to instantiate a Vector object in Manim with

the right arguments. This semantic grounding is what allows the system to generalize, handling everything from basic shapes to more sophisticated requests like “visually demonstrate eigenvector transformations.” Essentially, this module forms the backbone for the next step: actual code generation.

4.4.4. Code Generator Module (Generates Manim Code)

With the logical blueprint in hand, the Code Generator module gets to work, producing executable Manim Python scripts tailored to the user’s requirements. Here, templates, heuristics, and rule-based systems may be used to piece together the precise sequence of code needed to create the animation. The real benefit is that the user never has to worry about syntax errors, Python imports, or Manim’s sometimes overwhelming API surface. Whether the request is simple or elaborate, this module ensures the output code is both syntactically valid and semantically accurate. For instance, if someone asks for “a fading transition between two functions,” the generated script will correctly use Manim’s `FadeTransform` objects and animation timings.

4.4.5. Code Validator and Syntax Checker

Before any code is rendered, we run it through a robust validation pipeline. This is a crucial safeguard. The Validator checks for Python syntax errors, missing imports, and API misuse specific to Manim. But it doesn’t stop there, it can even catch logical inconsistencies or flag unsupported features before the rendering process starts, providing helpful feedback if something is wrong. This step not only prevents crashes and wasted computational resources but also ensures that the user receives a working animation every time. It’s like a spell-check for code, dramatically improving system reliability and user satisfaction.

4.4.6. Manim Renderer Engine

Once the code passes validation, the Manim Renderer Engine comes into action. This is the stage where code turns into captivating visuals. The engine interprets every line of the generated script, orchestrating the timing, sequencing, and layering of all objects on the screen. Manim’s high-performance vector graphics capabilities ensure that even intricate mathematical scenes, like 3D surfaces, parametric curves, or animated proofs are rendered crisply and smoothly. This module handles all the computational heavy lifting, abstracting away the technical details so the user can focus on the message, not the mechanics.

4.4.7. Output Video File

Finally, the completed animation is exported as a high-quality video file. This file is the end product that can be replayed, embedded in presentations, uploaded to e-learning platforms, or simply used for personal study. By providing the result in a universal video format, we make sure it is accessible across devices and software. The final animation precisely reflects the user's original intent, offering a seamless bridge from abstract idea to tangible visualization. For teachers, it's a ready-made teaching aid; for students, an interactive study tool; and for content creators, a powerful way to explain complex concepts visually.

In summary, this stepwise pipeline transforms raw, intuitive ideas into polished, pedagogical animations, no programming knowledge required. Each module is designed to minimize friction, maximize interpretability, and guarantee consistent, high-quality results. By thoroughly validating every stage and leveraging the power of AI and Manim, our system democratizes mathematical animation for everyone, making advanced educational content creation as easy as typing a sentence.

4.5. Tools and Technologies Component

This section explains in detail all the major tools and technologies that we have used throughout our project. It covers frameworks and libraries used for training and fine-tuning large language models (LLMs), generating and rendering animations, managing video and files, creating the user interface, and evaluating the overall system performance.

Table 4-1: Tools and Technologies

Components	Tools/Technologies
LLM finetuning	Hugging face transformers, Pytorch
Code Generation Models	Phi-2, Code LLaMa, Mistral
Visualization Engine	Manim
GUI framework	Gradio/ Streamlit
File and Video Handling	OS/File APIs, ffmpeg, Shutil
Evaluation Metrics	Accuracy, Runtime, Visual Quality

4.6. Large Language Models (LLMs)

Large Language Models (LLMs) are one of the most important breakthroughs in the field of artificial intelligence and natural language processing. In simple terms, an LLM is a computer model that learns to read, process, and generate human language in a way that often feels quite natural. The “large” part means that these models are trained on huge collections of text, ranging from books and articles to websites and conversations, so that they can recognize and mimic the patterns of human writing and speaking.

LLMs work by reading billions of sentences and learning the relationships between words, phrases, and ideas. When someone gives it a prompt or a question, the model tries to reply with the most appropriate response, just like a human would. It does not “understand” language the way people do, but it gets really good at predicting what kind of text should come next. Popular examples of LLMs include OpenAI’s GPT models (like ChatGPT), Google’s BERT, and others.

4.6.1. Basic Architecture of LLMs

The backbone of most modern LLMs is the **transformer** architecture. Before transformers, older models processed text one word at a time, which made it hard to understand the full context of longer sentences. Transformers solve this by looking at all words in a sentence at the same time (in parallel) and figuring out how each word relates to every other word. The key concept here is **attention**: the model learns to focus more on the important words for each part of the input.

The transformer consists mainly of two parts:

- **Encoder:** This part reads the input (such as a question or a statement) and turns it into a mathematical form (vectors) that represents the meaning.
- **Decoder:** This part takes the encoded information and generates the output, building the answer or response one word (or token) at a time.

The transformer architecture processes input text through a series of well-defined steps, each with its own technical importance. Here is a more detailed breakdown of the entire process:

1. Tokenization:

The first step is tokenization, where the input sentence or paragraph is broken down into smaller units called *tokens*. A token can be a word, a sub-word (like “ing” or “un”), or even a single character, depending on the tokenizer used. For example, the sentence “The cat sat” might become three tokens: [“The”, “cat”, “sat”]. In many modern models, a technique called **Byte Pair Encoding (BPE)** is used, which splits rare words into more common sub-words, helping the model handle any kind of text, including spelling mistakes or unknown words.

2. Embedding:

After tokenization, each token is converted into a numerical form called a *vector*, using an embedding layer. This process is called **word embedding**. Here, a token like “cat” becomes a list of numbers (e.g., [0.14, -0.09, 0.22, ...]) that represents its meaning in the context of the model. These vectors are stored in an **embedding matrix**, which is learned during training. This step is crucial because computers only understand numbers, and these vectors allow the model to process the text mathematically while capturing relationships between words (for example, “cat” and “dog” end up with similar vectors).

3. Attention Mechanism:

The transformer introduces a special mechanism called **self-attention** (or simply “attention”), which is what makes it so powerful. At this stage, the model calculates how much attention each token should pay to every other token in the sentence. This is done using **query**, **key**, and **value** vectors for each token. The idea is that, for each word, the model asks “Which other words in this sentence are important for understanding me?” For example, in the sentence “The cat sat on the mat,” the word “sat” may focus more on “cat” than on “mat”. This mechanism allows the model to understand context and dependencies, no matter how far apart words are in the sentence. Multiple attention heads are used in parallel (**multi-head attention**), so the model can look at different relationships at the same time.

4. Encoder Layers:

The encoder part of the transformer consists of several identical layers, each containing a multi-head self-attention mechanism followed by a **feed-forward neural network**. The self-attention allows the encoder to build a deep understanding of the entire input sequence, while the feed-forward network adds extra learning power. There are also normalization steps (**layer normalization**) and shortcut connections (**residual connections**) to help with training stability and faster convergence. The output of the encoder layers is a set of context-rich vectors, where each vector now contains information not just about a single token, but also its relationship with every other token in the input.

5. Decoder Layers:

The decoder also consists of several identical layers, but it works a bit differently. Each decoder layer has:

- **Masked Self-Attention:** The decoder predicts the output one token at a time. Masking ensures that, when generating the next word, the model cannot peek at future words, keeping the process realistic.
- **Encoder-Decoder Attention:** This part allows the decoder to focus on relevant parts of the encoded input sequence, linking what is being generated to the original prompt.
- **Feed-Forward Neural Network:** This processes the attended information and prepares it for the next decoder layer.

The decoder builds up the output sequence step by step, using both what it has generated so far and what it learned from the input.

6. Output Generation:

Finally, the output from the last decoder layer is passed through a **linear layer** and a

softmax function. The linear layer converts the decoder’s vectors into a score for each possible word in the model’s vocabulary. The softmax function then turns these scores into probabilities, and the model picks the word with the highest probability as the next token. This process repeats until the model produces a special end-of-sequence token, signaling that the output is complete. The sequence of tokens is then joined together to form the final answer, sentence, or (in the case of our project) generated code.

Overall, these steps allow the transformer model to deeply understand the input text, maintain context over long passages, and generate coherent, meaningful outputs. The technical advances in attention and parallel processing make transformers the backbone of modern large language models, enabling powerful applications like our Math2Visual system. All these steps happen in fractions of a second, which allows LLMs to quickly generate text that is coherent and relevant.

4.6.2. How LLMs Learn from Data

LLMs learn from data using a process called **supervised learning** or **self-supervised learning**. During training, the model sees millions or even billions of sentences and tries to predict the next word, fill in blanks, or rewrite sentences. If it makes a mistake, the model adjusts its internal parameters using a technique called *backpropagation*. Over many repetitions, it gets better and better at making accurate predictions. Training LLMs requires massive amounts of data and computing power.

4.6.3. Role of LLMs in Math2Visual

In our Math2Visual project, LLMs play a central role. The main goal of our system is to take mathematical questions or statements written in plain language and automatically generate animated visual explanations. When a user types a prompt like “Show the graph of $y = x^2$ and highlight its vertex,” the finetuned LLM processes the natural language, understands the mathematical intent, and generates structured Python code specifically for the Manim animation engine.

The LLM acts as a translator between human language and code. It recognizes mathematical expressions, keywords (like “plot,” “highlight,” “vertex”), relationships between them and then arranges these into correct instructions for Manim. This allows our project, Math2Visual to convert most of the math statement into a visual animation without manual coding.

With the help of LLMs finetuned for our specific use case, our system can:

- Understand a wide range of mathematical language and symbols, even if expressed in different styles,
- Detect the intent behind user prompts and identify important mathematical features,
- Generate Python code that is compatible with Manim, which then creates the actual animation.

Without LLMs, this type of natural language to animation conversion would not be possible. LLMs make Math2Visual flexible, user-friendly, and highly effective for students, teachers, and anyone who wants to visualize mathematics interactively.

To sum up, Large Language Models are powerful tools that make it possible to bridge the gap between human communication and computer understanding. In our Math2Visual project, the LLM is the heart of the system, converting plain math questions into structured code for visual animations. This not only makes math more interactive but also helps learners grasp complex concepts much more easily.

4.7. Hugging Face Transformers

Hugging Face Transformers is one of the most popular open-source libraries for working with modern Natural Language Processing (NLP) and code generation models. Developed by Hugging Face, this library provides an extensive collection of pre-trained models, including Code LLaMA, Mistral, Phi-2, StarCoder, CodeGen, GPT, BERT, and many others. These models are accessible through a unified API, making it straightforward for both beginners and experts to experiment with state-of-the-art AI in their own projects.

The main aim of Hugging Face Transformers is to democratize access to large language models by providing easy-to-use tools for model loading, tokenization, fine-tuning, and inference. It supports a wide variety of architectures such as encoder-decoder, decoder-only, and encoder-only models. The library is framework-agnostic, supporting both PyTorch and TensorFlow, but in our project, we use the PyTorch ecosystem because of its flexibility and strong research support.

4.7.1. Evaluation of Hugging Face Transformers for LLM-Based Applications

For a project like ours, which focuses on converting natural language math prompts into executable code, Hugging Face offers several key advantages:

- **Access to state-of-the-art models:** Many of the world's best open-source LLMs are

available on the Hugging Face Model Hub, ready to download and use.

- **Ease of fine-tuning:** The Trainer API abstracts away much of the complexity of training and fine-tuning large models, so we can focus on preparing high-quality datasets and evaluating results.
- **Tokenization and data processing:** Tokenizers for each model come ready out of the box, and can handle everything from plain English to mathematical formulas and code syntax.
- **Community and documentation:** The platform has a vibrant developer community and detailed documentation, which helps a lot whenever we face issues or want to follow best practices.
- **Integration with PyTorch:** All model components work natively with PyTorch, allowing us to use GPU acceleration for both training and inference.

How do we use it in our pipeline?

In our system, Hugging Face Transformers powers the entire model lifecycle: loading pre-trained LLMs, preparing our custom dataset (with pairs of math prompts and corresponding Manim code), tokenizing these pairs, running the fine-tuning loop, saving the adapted model, and using it to generate Manim code from new English inputs. This makes the whole development process systematic and less error-prone.

Step-by-step fine-tuning process with Hugging Face Transformers:

1. **Data Preparation:** We collect and clean a dataset consisting of natural language prompts (like “Plot the graph of $y = x^2$ ”) and their correct Manim code. This dataset is both manually curated and synthetically expanded using code templates.
2. **Tokenization:** We tokenize both the input (prompt) and output (code) using the specific tokenizer for our chosen model. This process converts human-readable text and code into integer sequences that the model can process.
3. **Model Loading:** We load a pre-trained base model from the Hugging Face Model Hub, such as Code LLaMA, Mistral, or Phi-2. The model checkpoint and matching tokenizer download automatically.
4. **Training (Fine-tuning):** We use the Hugging Face Trainer API (which wraps PyTorch’s DataLoader and training logic) to fine-tune the model on our dataset. The

model learns to map the tokenized English prompt to the correct tokenized Manim code by minimizing a cross-entropy loss function.

5. **Evaluation:** After training, we evaluate the model on a separate validation dataset. We use metrics like BLEU score (how similar is the generated code to reference code), execution success rate (does the code run in Manim without errors), and subjective checks for animation quality.
6. **Model Saving and Inference:** Once we identify the best-performing model, we save it and load it for inference, where it takes a new prompt and produces the corresponding Manim script.

Using such a flexible and powerful library gives our project a huge boost, saving us months of effort compared to building everything from scratch.

4.8. Code LLaMA

Code LLaMA is a specialized family of language models developed by Meta AI (Facebook AI Research), designed specifically for code generation and understanding tasks. Code LLaMA builds upon the original LLaMA (Large Language Model Meta AI) models but is further pre-trained and instruction-tuned on massive datasets of programming code and technical documentation. The first release of Code LLaMA came out in August 2023 and instantly gained popularity among researchers and developers working in software engineering, automation, and educational technology.

4.8.1. Architecture and Sizes

Code LLaMA follows a transformer decoder-only architecture, just like GPT-3, making it highly effective at generating code in a left-to-right, auto-regressive manner. The models come in various parameter sizes, including 7B, 13B, 34B, and 70B (B stands for billion parameters). Larger versions of Code LLaMA can capture more complex programming patterns and handle bigger contexts, but even the smaller sizes perform very well on common coding tasks and can run on standard university GPUs.

4.8.2. Training Data

Code LLaMA is trained on a carefully selected mixture of natural language text and billions of lines of publicly available source code from repositories like GitHub. Its training set includes a variety of programming languages such as Python, C++, Java, JavaScript, HTML,

and more, along with documentation and technical Q&A sites like Stack Overflow. This broad training allows Code LLaMA to understand not only programming syntax, but also the intent behind technical natural language prompts.

4.8.3. Role of Code LLaMA in Our Project

In our project, Code LLaMA acts as the main backbone for converting natural language math prompts into executable Manim code. Thanks to its extensive code-centric training, Code LLaMA can reliably translate even complex English instructions into correct Python scripts. Thus after finetuning it with manim specific dataset, it can help us create a accurate manim code as per user prompt. This means that when a user asks to “draw a parabola and mark its vertex,” Code LLaMA is able to generate the full code block required to create that animation.

Why Code LLaMA is suitable:

- **Open for research:** Code LLaMA is accessible to the academic community, making it practical for use in student projects and research labs.
- **Optimized for code:** Unlike general language models, Code LLaMA is tuned to produce valid, runnable code and to follow coding conventions.
- **Multilingual and multi-domain:** The model handles Python (used in Manim), as well as mathematical formulas and other programming languages, which is ideal for our broad range of math animation tasks.
- **Scalable:** Smaller Code LLaMA models like 7B and 13B run on affordable GPUs, so we can fine-tune and experiment in a university environment without massive resources.
- **Strong natural language understanding:** The model connects human intent to code, making it perfect for a system that takes English prompts and generates animations.

Overall, Code LLaMA gives us the flexibility, accuracy, and performance required to build an end-to-end pipeline for turning plain English mathematical ideas into Manim-powered animations, all in a way that is accessible to both students and educators.

4.9. Mistral

Mistral is a high-performance, open-weight large language model developed by Mistral AI, a European AI startup founded in 2023. Mistral models are designed to bring improvements in

speed, memory efficiency, and accuracy compared to previous open-source LLMs. Mistral also uses a decoder-only transformer architecture, similar to Code LLaMA.

The Mistral-7B model is the most widely used, but larger MoE versions also exist. Mistral models are trained on large-scale datasets that include English and multilingual text, technical documentation, and a lot of source code (Python, JavaScript, etc.). This makes Mistral especially suited for text-to-code tasks. **Model Innovations:**

- **Grouped Query Attention (GQA):** This mechanism allows Mistral to use less memory for attention calculations, which is often a bottleneck in LLMs. It enables longer context windows and faster inference.
- **Mixture of Experts (MoE):** Some Mistral models use an MoE design, where different “expert” sub-networks specialize in different data, boosting quality without dramatically increasing compute cost.

Benefits for Our Project:

- **Fast and memory-efficient:** We train and infer on moderate hardware, making it ideal for university labs and cloud resources.
- **Great at code synthesis:** Benchmarks show Mistral matches or outperforms models like LLaMA and Falcon in Python code generation.
- **Open-weight:** No licensing restrictions for academic use, and the community keeps improving model checkpoints.

4.10. Phi-2

Phi-2 is a lightweight yet high-performing language model developed by Microsoft Research and released in December 2023. While most LLMs aim for scale, Phi-2 shows that you can achieve great reasoning and code generation with a much smaller model. Phi-2 is also based on a decoder-only transformer architecture, highly capable for language and code tasks.

Phi-2 has about 2.7 billion parameters, but is trained on a very high-quality synthetic dataset focusing on textbook-level math, logical reasoning, and clean code samples. This means that although it is much smaller, it is much better at step-by-step reasoning and structured outputs compared to other models of similar or even larger size.

4.10.1. Design Philosophy

- **Quality over quantity:** By prioritizing high-quality, hand-crafted training data, Phi-2 “thinks” more logically than many models trained on random internet data.
- **Efficiency:** Its small size allows us to fine-tune and deploy even on laptops and low-end GPUs, which is practical for academic projects and quick prototyping.

4.10.2. Role of Phi-2 helps in our project

- **Lightweight:** Fast to train, quick inference perfect for rapid experimentation.
- **Reasoning and math focus:** The model is tuned for step-by-step solutions and precise logic, which is very helpful for generating correct Manim code from complex math descriptions.
- **Cost-effective:** It is ideal when we are working with limited hardware and budget being an open-sourced and light-weight model.

By working with Hugging Face Transformers as the backbone, and systematically experimenting with models like Code LLaMA, Mistral, and Phi-2, we build a robust pipeline for prompt-to-code translation. Each model brings its own strengths: Code LLaMA for large-scale understanding, Mistral for speed and code quality, and Phi-2 for logic and resource-efficiency. This experimentation allows us to select the best solution for generating accurate, runnable Manim code from user-friendly natural language prompts, making mathematical animations accessible for all.

4.11. Manim

One of the most important tools used in the Math2Visual project is the **Manim**. Manim, which stands for Mathematical Animation Engine, is an open-source Python library designed specifically for creating high-quality mathematical animations. The main motivation behind choosing Manim for our project is its power to transform abstract mathematical concepts into clear, visually engaging animations, all through Python code. This ability directly supports the goal of Math2Visual, which aims to take a simple natural language prompt related to mathematics and automatically generate a corresponding animated video—removing the need for users to write any animation code themselves.

In our experience as an engineering student, we have often found that certain mathematical ideas are much easier to understand when presented visually. For instance, reading equations

or imagining the behavior of a function graph can sometimes be confusing, especially for students new to the topic. Manim makes this process much more straightforward. It allows us to show, for example, how the graph of $y = x^2$ looks, how its vertex is positioned, or how an algorithm sorts data step by step all in the form of dynamic animations. In the context of Math2Visual, when a user provides a prompt like “Show the graph of $y = x^2$ and highlight its vertex,” our system processes the natural language and generates Manim code that produces the required animation. This bridges the gap between user-friendly input and professional-level educational content.

Manim operates by letting users script animations in Python. The animation is structured as a sequence of scenes, similar to slides in a presentation. Within each scene, we can add elements like text, geometric shapes, graphs, and equations, and animate their behavior over time. Unlike traditional video editing, there is no need to move objects around manually; instead, we simply specify what should happen, and Manim handles all the frame-by-frame details. For Math2Visual, this means once the user’s math prompt is converted to code, a polished animation video can be automatically rendered with minimal human intervention.

A significant advantage of Manim is its flexibility and the professional quality of its output. Animations produced with Manim support mathematical symbols, LaTeX equations, coordinate axes, function graphs, and even interactive stepwise explanations, which are extremely useful in an educational context. Since Manim is free and open-source, it is also ideal for student projects. we do not have to worry about license costs or software restrictions.

In summary, Manim serves as the core rendering engine for Math2Visual. It enables the project to convert natural language prompts into understandable, visually appealing, and accurate mathematical animations. This greatly enhances the educational value of the content generated by our system and makes complex mathematical concepts accessible to a wider audience. In my opinion, integrating Manim not only raises the standard of our project, but also demonstrates the potential of combining natural language processing with state-of-the-art visualization tools in the field of education.

4.12. GUI Implementation

A desktop GUI will be developed using Streamlit or Gradio (desktop mode), offering the following features:

- Code output area.
- Animation video player.

- Option to copy or edit or export code.

4.12.1. Python GUI

We used python GUI instead of traditional frontend and backend due to :

- i. Reduction of deployment complexity.
- ii. Enables full offline functionality.
- iii. Simplifies integration with Manim (already Python-based).
- iv. Offers lightweight and fast interaction.

4.13. Evaluation and Testing Strategy

Evaluation is a critical component of the system development process, especially for applications involving artificial intelligence, natural language processing, and automated code generation. For the *Math2Visual* system, it is essential to assess not only whether the generated Manim code is syntactically and semantically correct, but also whether the rendered animation aligns with the user's intended mathematical concept. This section outlines the strategies and metrics adopted to rigorously test the system's functionality, reliability, and usability.

4.13.1. Code Accuracy

This metric evaluates whether the generated Manim code:

- Compiles successfully without syntax or runtime errors.
- Follows correct Manim syntax conventions.
- Produces the intended animation without requiring manual correction.

Each generated code snippet is automatically parsed and tested in a sandbox environment before animation rendering. Logs are maintained to track compilation success rates and error types.

4.13.2. Animation Correctness

Even if code compiles, it is essential to verify whether the output animation visually reflects the meaning of the input prompt. Evaluation is conducted by:

- Human evaluators assessing how accurately the animations reflect the given input prompts.
- A checklist evaluating visual correctness (e.g., proper axes, labels, curve shape, annotation).
- Rating animations based on fidelity to the intended mathematical concept.

4.13.3. Latency

System responsiveness is another key factor, particularly in educational tools. Latency is measured in two stages:

1. **Inference Latency:** Time taken by the LLM to generate the Manim code.
2. **Rendering Latency:** Time required by the Manim engine to produce the final video (e.g., MP4 or GIF).

These timings are recorded using timestamps and averaged across test cases.

4.13.4. Testing Metrics

To objectively measure the quality of the generated code and resulting animations in our project, Math2Visual, we use several well-known evaluation metrics. Each metric provides a different perspective on how well our system performs, from how closely the generated code matches the ideal output to whether the code actually runs correctly. The main metrics used are explained below in detail:

- **BLEU Score:**

BLEU (Bilingual Evaluation Understudy) is a widely used metric originally developed for evaluating machine translation, but it also works well for code generation tasks. BLEU measures how closely the generated code matches a reference (ground truth) code by comparing overlapping *n-grams* which are short sequences of tokens (like words or symbols) of length *n*. The more *n-grams* the generated code shares with the reference, the higher the BLEU score.

The BLEU score is calculated as:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \cdot \log p_n \right) \quad (4.1)$$

where:

- BP = Brevity Penalty (to penalize overly short outputs),
- N = maximum n-gram length (typically 4),
- w_n = weight for each n-gram (usually equal for all n-grams),
- p_n = precision for n-gram of length n (i.e., the fraction of n-grams in generated code that also appear in the reference code).

The BLEU score ranges from 0 to 1, where a value closer to 1 means the generated code is more accurate and similar to the reference code, while a value closer to 0 indicates low similarity.

The brevity penalty (BP) is computed as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases}$$

where c is the length of the generated code and r is the length of the reference code. This prevents the model from generating too short outputs just to maximize n-gram precision.

Limitations of BLEU Score: While BLEU is useful for measuring n-gram similarity, it does not consider whether the code is functionally correct or executable. It may also penalize valid code that uses different variable names or structure, and it cannot detect logical errors or differences in code meaning.

In our project, we use BLEU to compare the generated Manim code with manually written reference code for the same math prompt. A higher BLEU score means our generated code is more structurally and semantically similar to the ideal code.

- **Edit Distance:**

Edit Distance (commonly referred to as Levenshtein Distance) is a metric used to measure how many changes are needed to convert the generated code into the reference (ideal) code. The changes can be character-level or token-level (such as lines or words), and the allowed operations are insertions, deletions, or substitutions. This metric is practical because it tells us how much effort it would take to manually fix the generated code if it is not already correct.

The Edit Distance is formally defined as:

$$\text{Edit Distance}(S, T) = \text{Minimum number of edits needed to transform string } S \text{ into } T \quad (4.2)$$

where:

- S = the generated code,
- T = the reference (ground truth) code.

The Edit Distance value ranges from 0 to infinity, where a value closer to 0 means the generated code is almost identical to the reference code, and higher values indicate more differences or required corrections.

Limitations of Edit Distance: This metric focuses on surface-level similarity and does not account for whether code with small edits actually works as intended. Also, different ways of writing functionally correct code may lead to a higher edit distance even if the outputs are the same. Edit distance does not check for actual correctness or successful execution.

In our project, we compute the edit distance between the generated Manim code and the reference code for each test case. A lower edit distance means our model’s output is closer to the ideal solution and requires less post-processing.

- **Execution Success Rate:**

Execution Success Rate evaluates the practical usability of the generated code by checking if it runs successfully without syntax or runtime errors. This metric is especially important for code generation tasks because even if the code looks correct, it must also be functional and produce the expected result or animation.

It is calculated as:

$$\text{Execution Success Rate} = \frac{\text{Number of successful executions}}{\text{Total number of generated samples}} \times 100\% \quad (4.3)$$

where:

- Number of successful executions = count of code samples that compile and run without error,
- Total number of generated samples = total code samples evaluated.

The Execution Success Rate ranges from 0% to 100%, where a value closer to 100% means most generated code runs correctly, while a value closer to 0% indicates frequent errors or failures.

Limitations of Execution Success Rate: This metric only tells us if the code runs without error, but does not check if the produced output or animation is actually correct or matches the user’s intent. A code sample might run but still generate the wrong result or a blank animation.

In Math2Visual, we use this metric to report the percentage of generated Manim code samples that compile and render the expected animation video without any errors. A higher execution success rate shows that our model is reliable and practical for end users.

4.13.5. Test Cases

A range of test prompts were used to validate system performance. Below are selected examples:

Table 4-2: Example Test Cases for Evaluation

Prompt (Input)	Expected Output (Math)	Success/Failure
Plot the graph of $y = \sin(x)$ from $-\pi$ to π	Smooth sine wave curve, labeled axes	Success
Visualize the unit circle with angle $\theta = \frac{\pi}{4}$ marked	Circle with radius 1 and an angle line showing $\frac{\pi}{4}$	Success
Draw $y = \frac{1}{x}$ and highlight vertical asymptote at $x = 0$	Hyperbola, vertical line at $x = 0$	Partial Success (asymptote not rendered)
Animate the derivative of $y = x^2$	Show original curve and its tangent moving across x-axis	Success
Show integral of x^2 from 0 to 2	Curve $y = x^2$ and shaded area under curve from 0 to 2	Success

Each test case is reviewed for both technical correctness and visual clarity. Notes are maintained to log partial errors, such as missing labels, incorrect axes, or incomplete animations.

5. FEASIBILITY ANALYSIS

5.1. Economic Feasibility

The project is economically viable, as it leverages open-source tools such as Manim, Hugging Face LLMs, and Python libraries, eliminating licensing costs. Development can be carried out using personal hardware, requiring minimal additional investment.

5.2. Technical Feasibility

The system is technically feasible with the current availability of open-source large language models (LLMs) and visualization tools. Python GUI frameworks like Gradio or Streamlit along with Manim CE, can be integrated on mid-range computing systems commonly available to students.

5.3. Operational Feasibility

The application is designed to be user-friendly and accessible for students and educators, enabling intuitive interaction through plain English prompts. The system meets its operational goals by delivering accurate animated outputs with minimal user effort, promoting efficient math learning.

5.4. Ethical Feasibility

The project is ethically sound as it uses open-source tools and publicly available models, ensuring transparency and responsible use of AI. No personal or sensitive data is collected during use. The system is designed for educational purposes only, helping users learn mathematical concepts through visual explanations without risk of misuse or bias.

6. EXPECTED OUTCOME

The successful implementation of the **Math2Visual: From Natural Language Prompts to Manim Code for Animated Video Generation** project is expected to result in a system capable of accurately translating natural language prompts related to mathematical concepts into executable Manim code. By fine-tuning open-source large language models such as Code LLaMA and Mistral, the system aims to generate high-quality Python scripts that can be automatically rendered into educational animations. This will significantly reduce the time, effort, and technical knowledge traditionally required to create math visualizations, making it accessible for both educators and learners with limited programming skills.

Furthermore, the project will enable detailed comparative analysis of different LLMs based on training loss, code quality, and output effectiveness. The system's performance will provide insights into the practical viability of fine-tuned models in domain-specific code generation tasks. Ultimately, this research is expected to contribute to the growing field of AI-powered educational technologies by offering a tool that simplifies and enhances the teaching and learning of complex mathematical topics through dynamic, visual content.

6.1. Prompt vs. Generated Code Example

The following example illustrates a basic prompt processed by the *Math2Visual* system and the corresponding Manim code generated to create a simple animation. When the output code from the model is compiled, it generates the mathematical animated video according to the prompt provided as shown in the figure below.

6.1.1. Input Prompt

"Plot the graph of $y = x^2$ and mark the vertex."

6.1.2. Mathematical Interpretation

$$y = x^2 \quad \text{with vertex at } (0, 0) \tag{6.1}$$

6.1.3. Generated Manim Code

```
from manim import *

class SimpleParabola(Scene):
```

```

def construct(self):
    # Create basic 2D axes
    ax = Axes(
        x_range=[-5, 5, 1],
        y_range=[-5, 10, 1],
        axis_config={"color": WHITE}
    )

    # Add axis labels using Text() instead of LaTeX
    x_label = Text("x").scale(0.5).next_to(ax.x_axis.get_end(), RIGHT)
    y_label = Text("y").scale(0.5).next_to(ax.y_axis.get_end(), UP)

    # Plot the function  $y = x^2$ 
    parabola = ax.plot(lambda x: x**2, color=BLUE)

    # Mark the vertex at (0, 0)
    vertex = Dot(ax.coords_to_point(0, 0), color=RED)
    vertex_label = Text("Vertex").scale(0.4).next_to(vertex, DOWN)

    # Animate the scene
    self.play(Create(ax))
    self.play(Write(x_label), Write(y_label))
    self.play(Create(parabola))
    self.play(FadeIn(vertex), Write(vertex_label))
    self.wait(2)

```

This output demonstrates the system's ability to convert a natural language prompt into a working Manim animation script. The graph of $y = x^2$ is plotted, the axes are labeled using plain text, and the vertex at the origin is marked with a red dot and a textual label.

6.1.4. Generated video after compilation of code

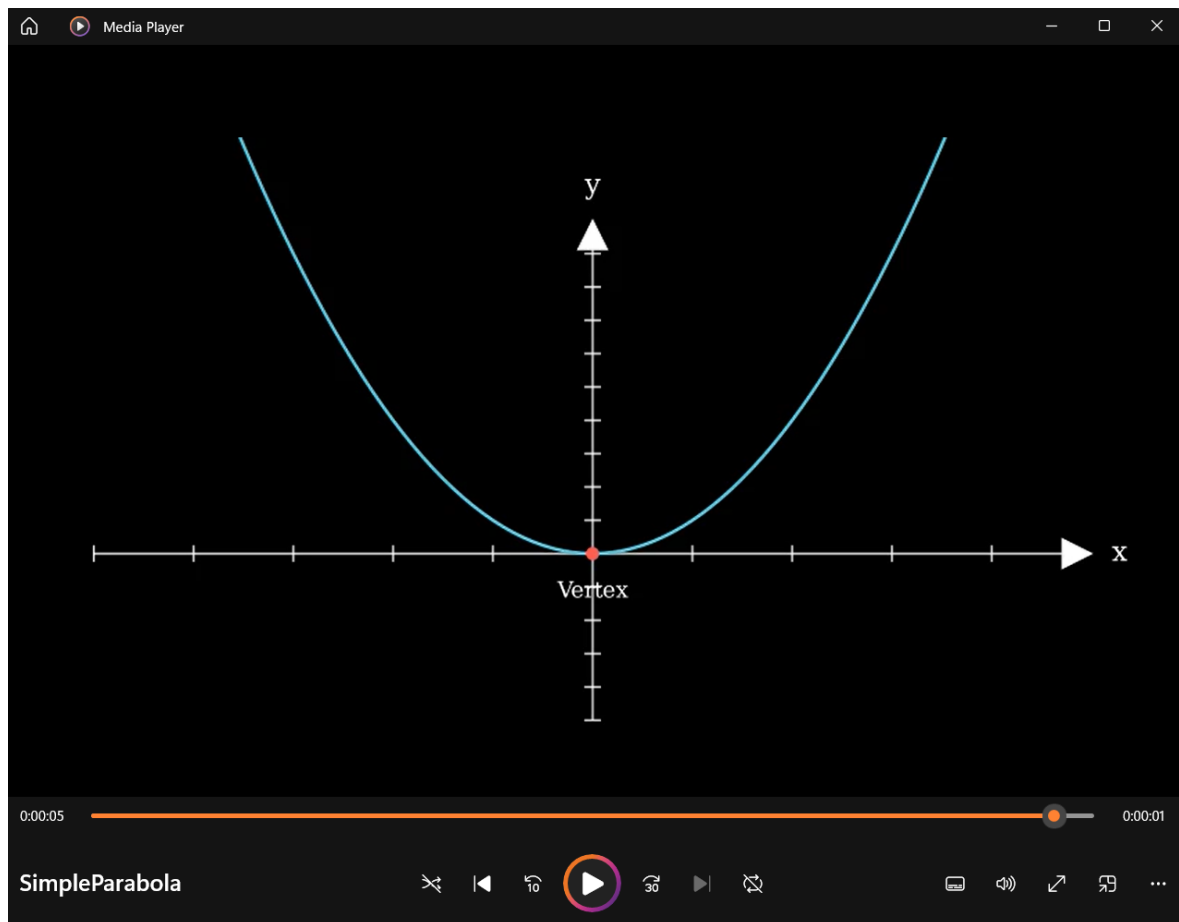


Figure 6-1: Rendered output of the animation generated from prompt

6.1.5. Wireframe

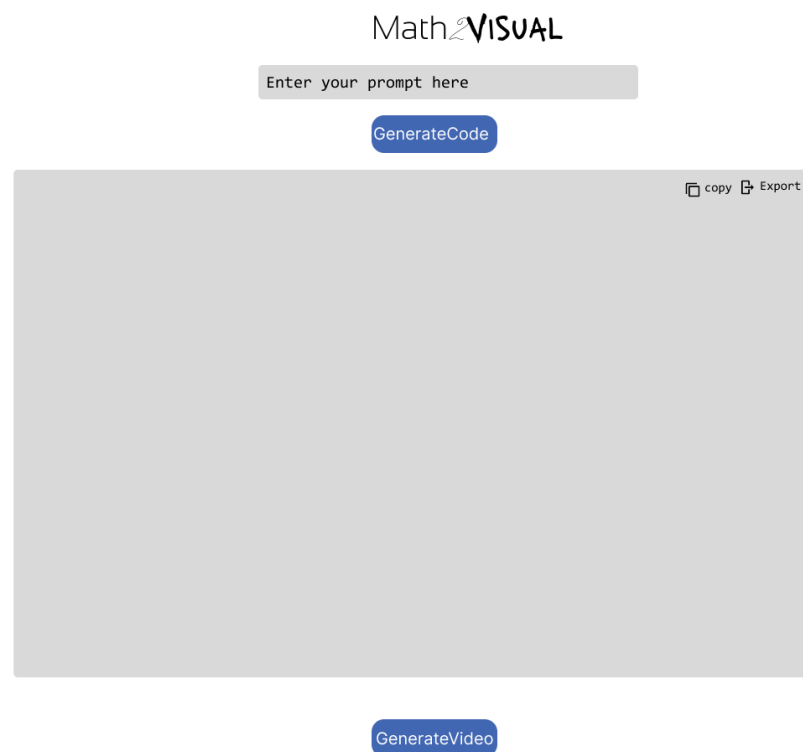


Figure 6-2: Wireframe of the Math2Visual interface showing the code generation panel

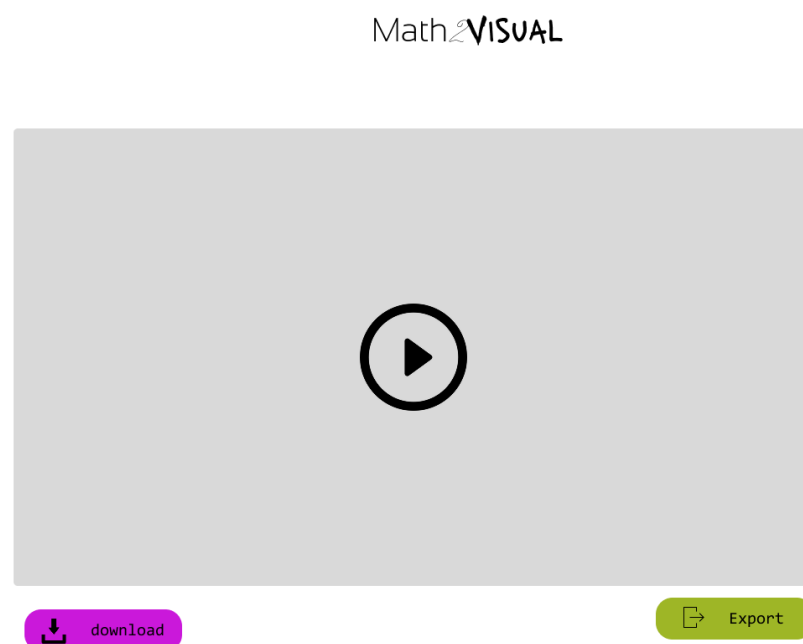


Figure 6-3: Wireframe of the Math2Visual interface displaying the video playback panel

7. APPENDICES

Appendix A: Gantt Chart

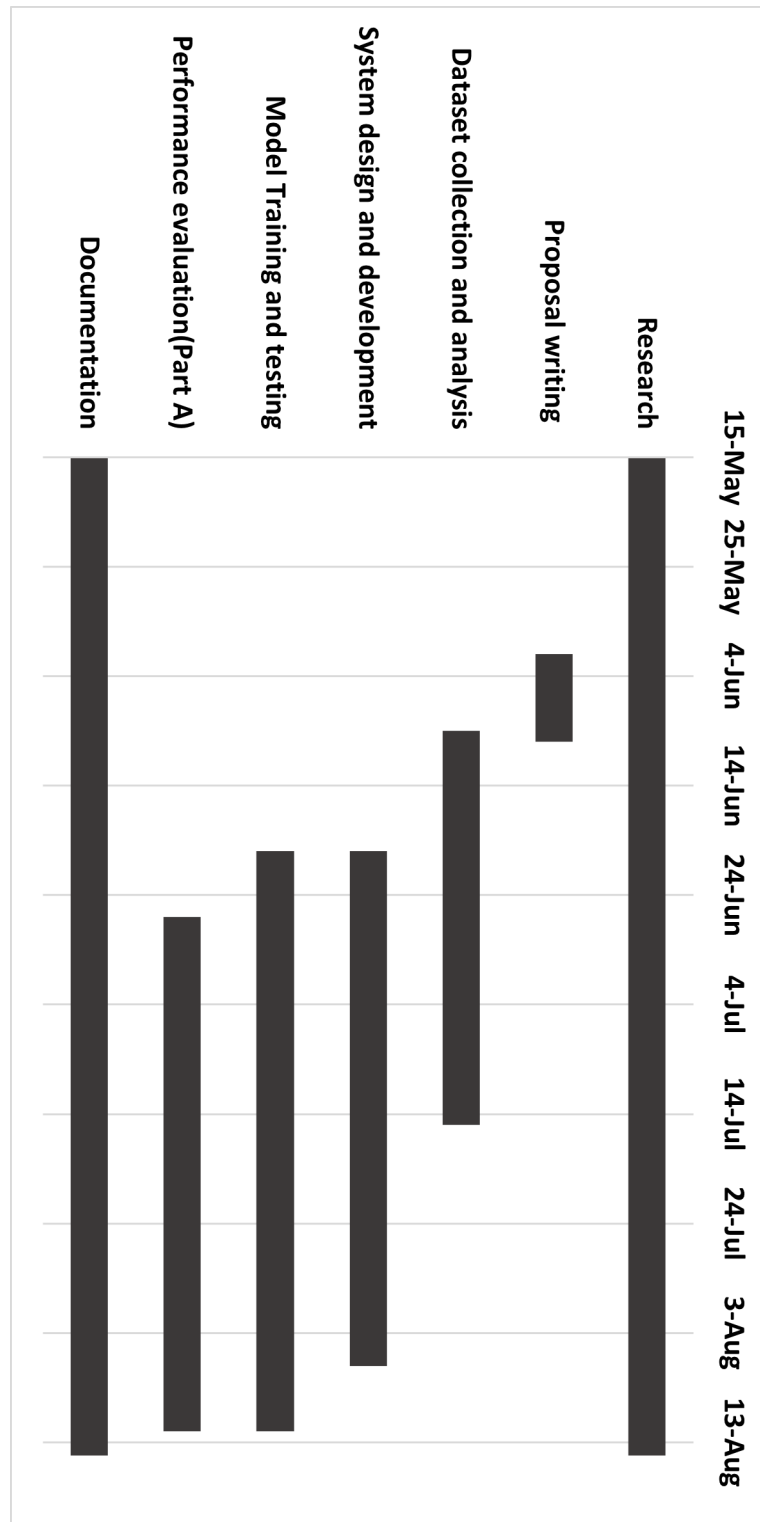


Figure 7-1: Gantt chart

Appendix B : Expected Budget

Table 7-1: Budget Estimation of the project

Components	Pricing (in NRs)
Google collab Pro (For GPU Access)	10,000
Cloud Storage (Hugging face or Open source LLM hosting)	5,000
Documentation	2,500
Miscellaneous	2,500
Total	20,000

Appendix C: Mathematical Foundation of Transformer-Based LLMs

Our main goal in this project is to convert natural language math prompts into animated Manim code. For this, we use transformer-based large language models (LLMs) like Code Llama and Mistral. These models depend on various mathematical operations from linear algebra, probability, and calculus. This section outlines the key mathematical concepts and operations that govern how these models process and generate sequences, forming the basis for their fine-tuning in our project.

1.Token Embedding

Each input token x_i (such as a word or symbol) is mapped to a dense vector:

$$\mathbf{e}_i = \text{Embedding}(x_i) \in \mathbb{R}^{d_{\text{model}}} \quad (7.1)$$

where x_i is the i -th token, and d_{model} is the embedding size.

2.Positional Encoding

To encode order, sinusoidal positional encodings are added:

$$\text{PE}_{(p,2k)} = \sin\left(\frac{p}{10000^{2k/d_{\text{model}}}}\right) \quad (7.2)$$

$$\text{PE}_{(p,2k+1)} = \cos\left(\frac{p}{10000^{2k/d_{\text{model}}}}\right) \quad (7.3)$$

where p is token position, k is the dimension index.

3.Query, Key, and Value Projections

Each embedding is projected to Query (Q), Key (K), and Value (V) spaces:

$$Q = EW^Q, \quad K = EW^K, \quad V = EW^V \quad (7.4)$$

where E is the embedding matrix, and W^Q, W^K, W^V are learnable weight matrices.

4.Scaled Dot-Product Attention

The core attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (7.5)$$

where d_k is the dimension of keys/queries, and softmax is applied row-wise.

5. Multi-Head Attention

Multiple attention heads allow the model to capture diverse patterns:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (7.6)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (7.7)$$

where h is the number of heads, and W^O is the output projection.

6. Masking (for Autoregressive Decoding)

For code generation, the model cannot attend to future tokens:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (7.8)$$

$$\text{Masked Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V \quad (7.9)$$

where M is the mask matrix.

7. Layer Normalization

Normalization stabilizes and accelerates training:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \cdot \gamma + \beta \quad (7.10)$$

where x is the input vector, μ and σ are its mean and standard deviation, and γ, β are learnable parameters.

8. Feedforward Neural Network (FFN)

Each token passes through a two-layer neural network:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (7.11)$$

where W_1, W_2 are weights and b_1, b_2 are biases.

9. Output Layer and Softmax

The output probabilities over vocabulary are given by:

$$P(y_i|x) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (7.12)$$

where z_i is the logit for token i .

10. Cross-Entropy Loss

Training minimizes the following loss:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^V y_i \log(\hat{y}_i) \quad (7.13)$$

where V is the vocabulary size, y_i is the ground truth (one-hot), and $\hat{y}_i$ is the predicted probability.

11. Backpropagation and Gradient Calculation

Model parameters are updated using gradients:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (7.14)$$

where w is a parameter, and z is any intermediate variable.

12. Gradient Descent Update

Each parameter is updated as:

$$w := w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w} \quad (7.15)$$

where η is the learning rate.

13. Tokenization

For completeness, tokenization splits the prompt into tokens:

$$T(\text{prompt}) = [x_1, x_2, \dots, x_n] \quad (7.16)$$

where T is the tokenizer and x_i are the resulting tokens.

All these mathematical steps from embedding, attention, and masking to normalization, loss minimization, and fine-tuning empower transformer-based LLMs for complex sequence modeling. By adapting them for Math2Visual, we automate converting mathematical language into Manim code, bridging the gap between human-readable math and animated visualization.

References

- [1] A. Arcavi, “The role of visual representations in the learning of mathematics,” *Educational Studies in Mathematics*, vol. 52, no. 3, pp. 215–241, 2003.
- [2] R. E. Mayer, “Cognitive theory of multimedia learning,” in *The Cambridge Handbook of Multimedia Learning*. Cambridge University Press, 2014, pp. 43–71.
- [3] 3Blue1Brown, “Manim documentation,” 2020, <https://docs.manim.community/en/stable/>.
- [4] ManimCommunity, “Manim,” 2023, <https://github.com/ManimCommunity/manim>.
- [5] A. Meurer, C. P. Smith, M. Paprocki *et al.*, “SymPy: symbolic computing in python,” *PeerJ Computer Science*, vol. 3, p. e103, 2017.
- [6] A. Radford, J. W. Kim, J. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” *arXiv preprint arXiv:2103.00020*, 2021.
- [7] OpenAI, “Openai codex,” 2021, <https://openai.com/blog/openai-codex>.
- [8] L. Tunstall, L. von Platen, and T. Wolf, *Natural Language Processing with Transformers*. O’Reilly Media, 2022.
- [9] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [10] M. AI, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [11] D. Hendrycks, S. Basart, C. Burns, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Natural language to code generation with transformers,” *arXiv preprint arXiv:2108.07732*, 2021.
- [12] Hugging Face, “Fine-tuning transformers,” 2023, <https://huggingface.co/docs/transformers/training>.
- [13] LangChain, “Natural language to python code,” 2023, <https://python.langchain.com>.
- [14] F. Khan, “Ai text to video model from scratch,” <https://github.com/FareedKhan-dev/AI-text-to-video-model-from-scratch>, 2024, gitHub repository.

- [15] F. Tan, S. Feng, and V. Ordonez, “Text2scene: Generating compositional scenes from textual descriptions,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 6710–6719.
- [16] OpenAI. (2024) Introducing sora: A text-to-video model. OpenAI. [Online; accessed 3-June-2025]. [Online]. Available: <https://openai.com/sora>
- [17] Google DeepMind. (2025) Veo: Our state-of-the-art video generation model. Google DeepMind. [Online; accessed 3-June-2025]. [Online]. Available: <https://deepmind.google/models/veo/>
- [18] Luma AI. (2024) Dream machine: Real-time text-to-video generation. Luma AI. [Online; accessed 3-June-2025].